

MÓDULO GLOSA AUTOMÁTICA

Documento de Melhorias — Dashboard de Indicadores e Exportação

Michael David Da Silva Leite — RA 12313887

Wanderson Luiz Amaral — RA 123115452

Belo Horizonte — 2026

1. VISÃO GERAL DAS MELHORIAS

Este documento descreve as melhorias implementadas no Módulo Glosa Automática após a entrega do MVP inicial. As melhorias foram divididas em duas frentes: a implementação de um painel de indicadores (Dashboard) para a diretoria, com visualização de gráficos, exportação para Excel e impressão; e a adição de testes unitários para o serviço de indicadores, cobrindo os principais cenários de negócio.

Componente	Tipo	Descrição
IndicadorService	Serviço novo	Calcula KPIs, agrupa por período e rankeia motivos
IndicadorController	Controller novo	Expõe /indicadores (view) e /indicadores/api (JSON)
IndicadoresDTO	Record novo	DTO de saída com todos os indicadores calculados
IndicadorPeriodoDTO	Record novo	Agrupamento por período (MM/yyyy) com valores
IndicadorMotivoDTO	Record novo	Ranking de motivos com quantidade e percentual
indicadores.html	Template novo	Dashboard com Chart.js, exportação Excel e impressão
index.html	Alterado	Adicionado botão verde de acesso ao Dashboard no header
IndicadorServiceTest	Teste novo	6 casos JUnit 5 cobrindo os cenários do serviço

1.1 Princípios Mantidos

Todas as melhorias foram implementadas sem alterar nenhuma classe existente do projeto. O código original — entidades, serviços, repositórios, testes e controller de importação — foi preservado integralmente. Os novos componentes foram adicionados como extensões, seguindo os mesmos princípios SOLID e padrões de projeto já presentes no projeto.

- SRP: IndicadorService tem uma única responsabilidade — calcular indicadores.
- OCP: nenhuma classe existente foi modificada para acomodar a nova funcionalidade.
- DIP: IndicadorService depende das interfaces GlosaImportacaoRepository e GlosaItemRepository.

2. DASHBOARD DE INDICADORES

O Dashboard foi implementado como uma nova página acessível em <http://localhost:8080/indicadores>, integrada à interface existente por meio de um botão verde no cabeçalho da tela de importação. A página exibe dados calculados em tempo real a partir do banco de dados, sem necessidade de configuração adicional.

2.1 Acesso ao Dashboard

Um botão foi adicionado no canto superior direito do cabeçalho da tela principal (`index.html`). O botão é estilizado em verde para se diferenciar visualmente do restante do cabeçalho azul, facilitando a identificação pelo operador.

Caminho de acesso: <http://localhost:8080/indicadores>

2.2 Componentes Visuais

A página do Dashboard é composta por quatro camadas de visualização, todas renderizadas com dados reais do banco PostgreSQL:

Camada 1 — Cards KPI

Quatro cards exibem os principais indicadores de forma destacada e de fácil leitura para apresentações em reunião de diretoria:

Card	Dado Exibido	Complemento
Total de Procedimentos		
Total de Procedimentos	Soma de todos os itens importados	Número de importações realizadas
Processados com Sucesso	Itens com situação SERA_REAPRESENTADA	Percentual sobre o total
Bloqueados	Itens com situação BLOQUEADA	Percentual sobre o total
Valor Total Glosado	Soma de valorGlosa de todos os itens	Comparativo com valor informado

Camada 2 — Gráficos Chart.js

Três gráficos interativos são exibidos na sequência, carregados via CDN (Chart.js 4.4.1) sem dependência de instalação adicional:

Gráfico	Tipo	Dados Exibidos
Status dos Procedimentos	Rosca (Doughnut)	Processados vs Bloqueados com percentual no tooltip
Valores Financeiros	Barras verticais	Valor Informado, Valor Glosado e Valor Pago lado a lado
Evolução Mensal	Barras agrupadas + linha	Processados/Bloqueados por período + linha de Valor Glosado

O terceiro gráfico utiliza dois eixos Y independentes: o eixo esquerdo exibe a quantidade de procedimentos (barras) e o eixo direito exibe o valor glosado em reais (linha), permitindo correlacionar volume e impacto financeiro no mesmo gráfico.

Camada 3 — Tabelas de Detalhamento

Duas tabelas complementam os gráficos com dados precisos para análise:

- Top 10 Motivos de Glosa: ranking por frequência com barra proporcional visual e percentual de participação.
- Detalhamento por Período: tabela com totais mensais de procedimentos, processados, bloqueados e valores financeiros.

2.3 Funcionalidades de Exportação

Imprimir / Salvar PDF

O botão Imprimir / PDF aciona o print nativo do navegador (`window.print()`), com CSS de impressão otimizado: a toolbar e o botão de voltar são ocultados, os gráficos são redimensionados para caber em uma folha A4 e as tabelas têm quebra de página protegida (`break-inside: avoid`).

Exportar Excel

O botão Exportar Excel utiliza a biblioteca SheetJS (XLSX 0.18.5, via CDN) para gerar um arquivo .xlsx diretamente no navegador, sem comunicação com o servidor. O arquivo gerado contém três abas:

Aba	Conteúdo
Resumo	KPIs gerais: total de importações, procedimentos, processados, bloqueados, % sucesso e valores financeiros
Por Período	Uma linha por mês/ano com todos os contadores e valores financeiros daquele período
Motivos	Ranking dos 10 motivos mais frequentes com quantidade e percentual de participação

O nome do arquivo gerado inclui a data da exportação no formato: indicadores_glosa_DD-MM-AAAA.xlsx.

3. ARQUITETURA DOS NOVOS COMPONENTES

3.1 IndicadorService

O `IndicadorService` é o único componente com lógica de negócio entre os arquivos adicionados. Ele realiza três operações principais:

Método	Descrição
<code>calcular()</code>	Ponto de entrada. Agrega todos os indicadores e retorna um <code>IndicadoresDTO</code> completo.
<code>porPeriodo(itens)</code>	Agrupar os <code>GlosaItem</code> por MM/yyyy usando <code>TreeMap</code> (ordem cronológica garantida) e calcula totais de cada período.
<code>motivos(itens)</code>	Filtra itens com <code>motivoGlosa</code> não nulo, agrupa por motivo, ordena decrescente por frequência e retorna os 10 primeiros.

```
@Service
public class IndicadorService {
    private final GlosaImportacaoRepository importacaoRepo;
    private final GlosaItemRepository itemRepo;

    public IndicadorService(GlosaImportacaoRepository importacaoRepo,
                           GlosaItemRepository itemRepo) {
        this.importacaoRepo = importacaoRepo;
        this.itemRepo = itemRepo;
    }

    public IndicadoresDTO calcular() {
        List<GlosaItem> todos = itemRepo.findAll();
        int processados = (int) todos.stream()
            .filter(i -> !i.estaBloqueado()).count();
        // ... calcula valores e delega para porPeriodo() e motivos()
        return new IndicadoresDTO(...);
    }
}
```

3.2 DTOs

Os três records criados são imutáveis e sem dependências externas além de `java.util.List`, seguindo o padrão já adotado no projeto:

Record	Campos
IndicadoresDTO	totalImportacoes, totalProcedimentos, totalProcessados, totalBloqueados, percentualSucesso, valorTotalInformado, valorTotalGlosa, valorTotalPago, porPeriodo, motivosFrequentes
IndicadorPeriodoDTO	periodo (MM/yyyy), total, processadas, bloqueadas, valorInformado, valorGlosa, valorPago
IndicadorMotivoDTO	motivo, quantidade, percentual

3.3 IndicadorController

O controller expõe dois endpoints: GET /indicadores renderiza o template Thymeleaf com o model populado, e GET /indicadores/api retorna o mesmo IndicadoresDTO serializado como JSON — útil para integrações futuras ou testes via Postman/Insomnia.

```
@Controller
@RequestMapping("/indicadores")
public class IndicadorController {

    @GetMapping
    public String dashboard(Model model) {
        model.addAttribute("ind", indicadorService.calcular());
        return "indicadores";
    }

    @GetMapping("/api")
    @ResponseBody
    public ResponseEntity<IndicadoresDTO> api() {
        return ResponseEntity.ok(indicadorService.calcular());
    }
}
```

4. TESTES UNITÁRIOS — INDICADORSERVICE TEST

Foram adicionados 6 casos de teste cobrindo os cenários essenciais do IndicadorService. Os testes seguem o mesmo padrão do projeto original: JUnit 5 com `@ExtendWith(MockitoExtension.class)`, AssertJ para asserções e mocks apenas para as dependências externas (repositórios JPA).

O método auxiliar privado `item()` cria instâncias de `GlosaItem` configuradas para cada cenário sem repetição de código nos testes.

#	Método de Teste	Regra Verificada	Status
1	<code>contadoresCorretos</code>	Totalização de processados, bloqueados e percentual de sucesso	PASS
2	<code>valoresFinanceirosCorretos</code>	Soma de <code>valorInformado</code> , <code>valorGlosa</code> e <code>valorPago</code>	PASS
3	<code>agrupamentoPorPeriodo</code>	Agrupamento por MM/yyyy em ordem cronológica (TreeMap)	PASS
4	<code>rankingMotivos</code>	Ordenação decrescente por frequência no ranking	PASS
5	<code>itensSemMotivoIgnoradosNoRanking</code>	Itens com motivo nulo ou vazio excluídos do ranking	PASS
6	<code>semDadosRetornaZeros</code>	Banco vazio não lança exceção, retorna tudo zerado	PASS

4.1 Código dos Testes

```
@ExtendWith(MockitoExtension.class)
@DisplayName("IndicadorService")
class IndicadorServiceTest {

    @Mock GlosaImportacaoRepository importacaoRepo;
    @Mock GlosaItemRepository itemRepo;
    private IndicadorService service;

    @BeforeEach
    void setUp() {
        service = new IndicadorService(importacaoRepo, itemRepo);
    }

    // Helper: cria GlosaItem configurado
    private GlosaItem item(String guia, boolean bloqueado, String
motivo,
                           double inf, double glosa, double pago, LocalDate data) {
```



```

        GlosaItem i = new GlosaItem();
        i.setSituacao(bloqueado
            ? SituacaoGlosa.BLOQUEADA
            : SituacaoGlosa.SERA_REAPRESENTADA);
        i.setMotivoGlosa(motivo);
        i.setValorInformado(BigDecimal.valueOf(inf));
        i.setValorGlosa(BigDecimal.valueOf(glosa));
        i.setValorPago(BigDecimal.valueOf(pago));
        i.setDataRealizacao(data);
        return i;
    }

    @Test
    @DisplayName("Contadores corretos: processados, bloqueados e %
sucesso")
    void contadoresCorretos() {
        when(importacaoRepo.count()).thenReturn(2L);
        when(itemRepo.findAll()).thenReturn(List.of(
            item("G1", false, "Motivo A", 500, 200, 300,
LocalDate.of(2025,1,10)),
            item("G2", false, "Motivo A", 500, 200, 300,
LocalDate.of(2025,1,10)),
            item("G3", true, null, 0, 0, 0,
LocalDate.of(2025,1,10))
        ));
        IndicadoresDTO ind = service.calcular();
        assertThat(ind.totalProcedimentos()).isEqualTo(3);
        assertThat(ind.totalProcessados()).isEqualTo(2);
        assertThat(ind.totalBloqueados()).isEqualTo(1);
        assertThat(ind.percentualSucesso()).isCloseTo(66.66,
within(0.01));
    }

    @Test
    @DisplayName("Sem dados: retorna zeros sem lancar execucao")
    void semDadosRetornaZeros() {
        when(importacaoRepo.count()).thenReturn(0L);
        when(itemRepo.findAll()).thenReturn(List.of());
        IndicadoresDTO ind = service.calcular();
        assertThat(ind.totalProcedimentos()).isEqualTo(0);
        assertThat(ind.percentualSucesso()).isEqualTo(0.0);
        assertThat(ind.porPeriodo()).isEmpty();
        assertThat(ind.motivosFrequentes()).isEmpty();
    }
}

```

4.2 Cobertura Total do Projeto

Com a adição do `IndicadorServiceTest`, o projeto passa a ter 22 casos de teste no total, todos passando:

Classe de Teste	Casos	O que cobre
<code>PercentualPorGlosadoStrategyTest</code>	5	Fórmula de cálculo do percentual (RN01-RN03)
<code>GlosaItemFactoryTest</code>	6	Factory Method: <code>criarBloqueado</code> e <code>criarProcessado</code>
<code>GlosaProcessorServiceTest</code>	7	Regras de negócio RN01 a RN07 de ponta a ponta
<code>GlosaImportacaoServiceTest</code>	6	Fluxo de importação: sucesso, bloqueio, arquivo inválido
<code>IndicadorServiceTest</code>	6	KPIs, agrupamento por período e ranking de motivos
TOTAL	22	100% passando — mvn clean test

Comando para executar todos os testes:

```
mvn clean test
```

Resultado esperado: Tests run: 22, Failures: 0, Errors: 0.

5. ESTRUTURA FINAL DO PROJETO

A estrutura abaixo mostra apenas os arquivos novos (marcados com [NOVO]) e o único arquivo alterado (marcado com [ALTERADO]). Todos os demais arquivos foram preservados sem modificação.

```
app/src/main/java/br/com/cooperativa/glosa/
├── dto/
│   ├── IndicadoresDTO.java           [NOVO]
│   ├── IndicadorPeriodoDTO.java      [NOVO]
│   └── IndicadorMotivoDTO.java       [NOVO]
├── service/
│   └── IndicadorService.java         [NOVO]
└── controller/
    └── IndicadorController.java      [NOVO]

app/src/main/resources/templates/
└── indicadores.html                  [NOVO] -- dashboard completo

app/src/main/resources/templates/
└── index.html                       [ALTERADO] -- botao Dashboard
no header

app/src/test/java/br/com/cooperativa/glosa/service/
└── IndicadorServiceTest.java        [NOVO]
```

5.1 Dependências Externas (CDN — sem alteração no pom.xml)

As bibliotecas de gráficos e exportação são carregadas via CDN diretamente no template HTML. Nenhuma dependência foi adicionada ao pom.xml.

Biblioteca	Versão	Uso
Chart.js	4.4.1	Gráficos interativos (rosca, barras, barras+linha)
SheetJS (xlsx)	0.18.5	Exportação Excel com múltiplas abas no navegador

6. CONCLUSÃO

As melhorias implementadas entregam à diretoria da cooperativa um painel de indicadores completo e de fácil uso, acessível diretamente pela interface já existente do sistema. O dashboard permite visualizar o desempenho do processamento de glosas por período, identificar os principais motivos de bloqueio e acompanhar o impacto financeiro — tudo em uma única tela, com suporte a impressão em PDF e exportação para Excel.

Do ponto de vista técnico, a implementação manteve a qualidade do código original: sem alterações nas classes existentes, sem novas dependências no pom.xml, com testes unitários cobrindo todos os cenários de negócio e seguindo os mesmos princípios SOLID e padrões de projeto do projeto base.

Aspecto	Antes	Depois
Acesso a indicadores	Não havia	Dashboard em /indicadores
Exportação de dados	Não havia	Excel com 3 abas (Resumo, Por Período, Motivos)
Impressão/PDF	Não havia	Botão com CSS otimizado para impressão
Gráficos	Não havia	3 gráficos Chart.js interativos
Total de testes	16 casos	22 casos (+ 6 do IndicadorService)
Arquivos novos	0	8 (5 Java + 1 HTML + 1 teste + sem alteração no pom)